Datacenter Optimization Project Final Report

James Faber

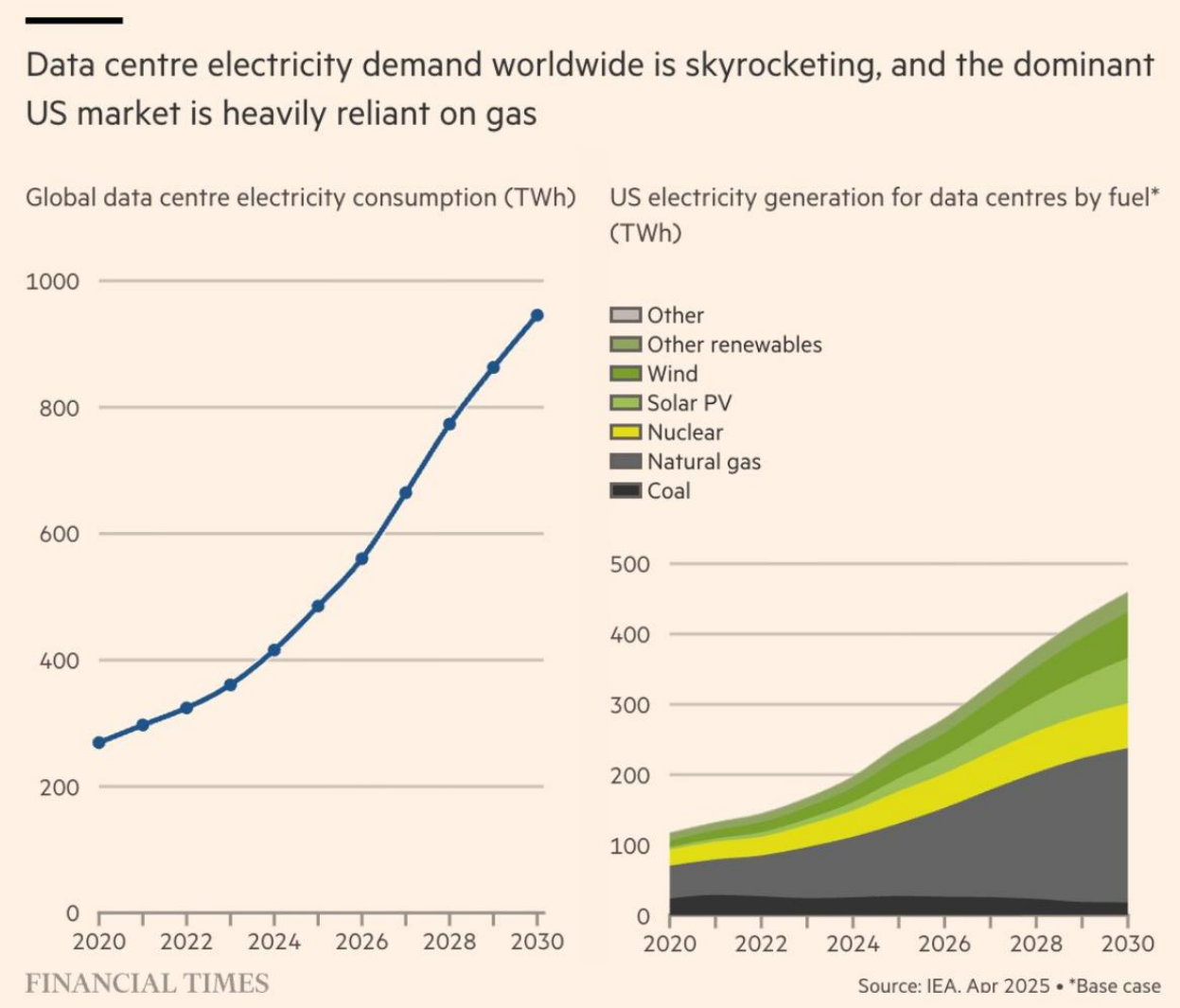
June 14, 2026

Table of Contents

Executive Summary

In 2026, hundreds of datacenters are being built around the world to handle the ever-increasing demand for AI. One of the negative aspects of this build-out that many people will be familiar with is the high electricity consumption that datacenters need to function. This creates problems because the price of electricity goes up where datacenters are built, and generating more power leads to more carbon emissions. But what if datacenter operations could be optimized to lower energy consumption instead?

This was the question I set out to answer at the beginning of this year; and as a practitioner of analytics, I believed that a solution was hiding in the data.



Luckily, there are a few publicly available datasets about datacenter operations. The most detailed dataset by far is the “MIT Supercloud Dataset” released by MIT in 2022, which can be accessed for free via Amazon Web Services (AWS). This dataset recorded individual “jobs” that were run on the MIT datacenter over a 9-month period. A job is just a program that was submitted to the datacenter for its supercomputers to process, and many of the jobs that were run at MIT were AI training models, which makes it especially useful. Despite being four years old at this point, there have not been any comparable datasets published since 2022, and the MIT Supercloud is still actively being used for research purposes.

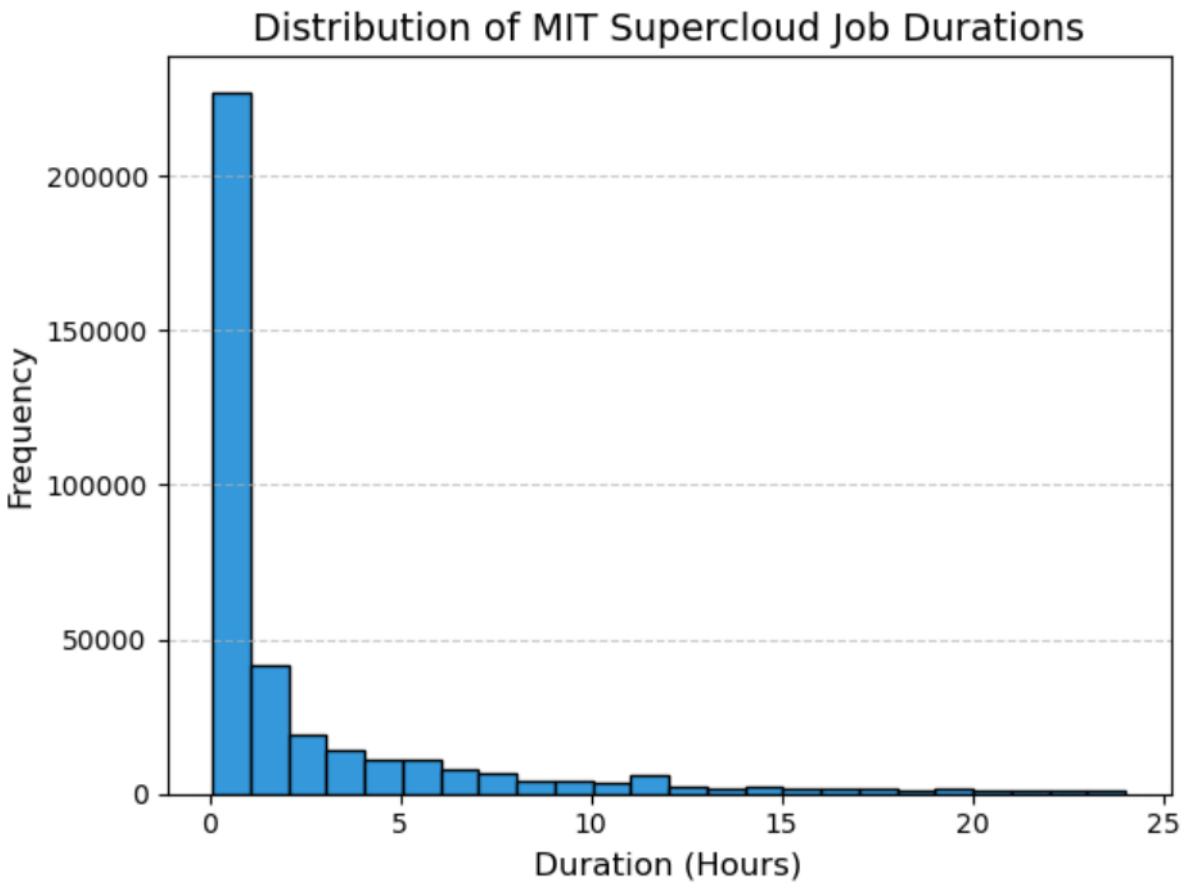
After exploring and analyzing the dataset, a key finding was that a whopping 20% of all jobs failed to finish properly or were manually cancelled before they could be completed. Since individual jobs can run for multiple hours or even days at a time, when they don't finish it is a waste of electricity that could have been used by a "good" job. Jobs that do not complete successfully will be referred to as "job failures" going forward for consistency.

This discovery led to the project idea; to build a machine learning model that can predict job failures well before they actually happen. If an accurate prediction can be made, then a job that is very likely to fail could be automatically stopped, which would save hours of electricity usage.

Project Scope

MIT's dataset is quite large at 2TB in size when fully uncompressed. It has a treasure trove of datacenter monitoring information due to its purpose of supporting academic research. On the GitHub page for this project, a copy of the original MIT Supercloud academic paper is provided for reference, which goes into detail about all the data that was collected. For this project, it was necessary to narrow the scope to something more manageable than the entire MIT repository.

The hypothesis for this project was that a prediction could be made about whether a job would fail by using monitoring information from the early stages of a job. After diving into the available data, it was found that jobs with longer durations had a higher likelihood of failure compared to jobs with shorter durations. The project scope was thus narrowed to focus exclusively on jobs that ran for at least 2 hours, based on the assumption that 2 hours of data would be sufficient to make a prediction, and given the large sample size of 15,000 job failures in this subset. As can be seen from the chart below, the majority of jobs in the MIT Supercloud dataset ran for less than 2 hours, so the choice to only focus on "over 2-hour jobs" significantly refines the scope.



NOTE: there are also jobs that exceeded 24 hours of runtime, but these were excluded from the above graph to improve readability.

70% of jobs in the "over 2-hour" subset completed successfully, so there are plenty of "good" jobs to compare to the jobs that failed. In addition to the 15,000 job failures, the subset also includes 22,000 jobs that were manually cancelled by the user who submitted them. While these cancelled jobs also represent a waste of electricity, modeling human behavior is outside the scope of this project. In other words, the predictive machine learning model was trained strictly using job failures and job successes.

Since the model would be performing a binary classification for each job (Success or Failure), the best practice is to train the model using a balanced dataset. What this means is that having a huge amount of successful jobs drowning out the minority of jobs that failed is undesirable. For the final dataset, the 15,000 jobs that failed were kept, and a random sample of 15,000 jobs that were completed successfully was taken to achieve an even 50-50 split. Going forward the analysis is concerned with 30,000 total jobs, which is

less than 10% of the entire MIT repository. As a reminder, all of the 30,000 jobs in the final dataset had a duration of over 2 hours.

Lastly, it would be remiss not to mention that in actual fact, there is not a single "Job Failure" end-state for a job. There are multiple different ways that a job can fail, including Timeout error, Out-of-Memory error, and General Crash. Each of these failure types are unique, however for this project they are grouped together so that the model can predict a binary outcome (Success or Failure). Further details about the reasoning for this decision is provided in the Jupyter Notebooks on the project's GitHub page.

Data Used

What kind of datacenter monitoring information could predict a job failure, one might ask?

All modern personal computers contain both a Central Processing Unit (CPU) and Graphics Processing Unit (GPU). A CPU handles general tasks and instructions for the system, while a GPU specializes in rapidly processing visuals and graphics. Datacenters contain thousands of powerful and interconnected CPU's and GPU's that are used to run jobs such as AI training workloads.

For each job in the MIT dataset, there exists a unique log file with CPU timeseries data collected at 10 second intervals. Timeseries data includes information such as the CPU Utilization (how hard the CPU is being worked) by the job, and how much memory it's using. Likewise, for just under half of the jobs, there exists a unique GPU timeseries log file with data collected at 0.1 second intervals. GPU's are not allocated for all jobs because they are only used for specialized tasks which require high-throughput arithmetic.

Step	Node	Series	ElapsedTime	EpochTime	CPUFrequency	CPUTime	CPUUtilization	RSS	VMSize	Pages	ReadMB	WriteMB
batch	r8937440-	0	0	1623532271	1226	0	0	4748	201088	0	0	0
batch	r8937440-	0	10	1623532281	1226	14.36	143.6	1200880	20556980	7	47.28492	8.128551
batch	r8937440-	0	20	1623532291	1226	24.85	248.5	2654708	54341504	7	252.7403	0.127806
batch	r8937440-	0	30	1623532301	1226	23.49	234.9	5778808	57747952	7	1086.672	0.003768
batch	r8937440-	0	40	1623532311	1226	40.59	405.9	6086164	58010096	7	665.8463	0.000639
batch	r8937440-	0	50	1623532321	1226	40.55	405.5	6229572	58042868	7	663.9005	0.000544
batch	r8937440-	0	60	1623532331	1226	40.24	402.4	6356984	58305012	7	654.0058	0.000553

The above image is an example CPU Timeseries log showing data collected from the first minute a job ran. Note that the "CPU Utilization percentage" is inflated by the number of CPU cores allocated to each job.

Now, since each job potentially has thousands of rows of CPU and/or GPU timeseries data, it was not realistic to train the predictive model using the full log files. The approach taken was to create simple summary statistics about the CPU & GPU data for the first 2 hours that each job was running. For example, one summary statistic I created was the Median CPU Utilization percentage within the first two hours a job ran. The hope was that these statistics would make good predictor variables for determining if a job would succeed or fail.

Since it was not inherently known what specific monitoring data could effectively predict a job failure, over 50 new predictor variables (also called "features") were created that could

feasibly do this. Many of these features were the maximum, minimum, median, and Inter-Quartile Range statistics for data that existed in the CPU/GPU logs. Complex features were also created, such as the rate of change (or slope) of a job's memory usage. A full data dictionary can be found on the project's GitHub page.

Engineering the Final Dataset

Even though the project scope had been refined to include less than 10% of the MIT Supercloud repository, a problem remained. Processing those 50+ new features for each one of the 30,000 jobs in the subset presented a significant data engineering challenge.

This required finding the CPU and GPU timeseries logs for each job within the AWS S3 bucket that the MIT data was stored in, which was not easy because the logs were stored as CSV files within unmarked folders. Cleaning and Processing each log file also took time, as they could often be thousands or tens of thousands of rows each. The computing power needed to accomplish this task was more than my 6-year-old laptop was able to handle.

The answer to this problem was Databricks. Databricks is a cloud-based platform which specializes in processing large volumes of data, fast. It does this by utilizing the Apache Spark engine and seamlessly connecting to other cloud platforms such as AWS. To take advantage of this powerful framework, the SQL code written to originally create the new features was converted to PySpark code, and Databricks was able to handle the processing without relying on local computing resources. It was also possible to fit all 30,000 jobs within the Databricks free-tier plan by splitting the job list into 4 pieces and running one quarter at a time to get around the usage limits.

Once all the processing had been completed, the result was a new 30,000 row table which included metadata about each unique job, and a value for each of the 58 engineered features (if one existed). Not all jobs used a GPU for example, so variables based on GPU statistics would be blank for those jobs. All of the PySpark code written for this portion of the project is published on the project's GitHub page and includes comments. The final dataset is also available on GitHub as a CSV file.

Model Training & Results

Now for the fun part, building a Machine Learning (ML) model to see if the hypothesis holds- or if this was just a crazy idea.

To begin, a random 20% chunk of the final dataset was set aside for model validation and testing, and the remaining 80% was used to train the model. Out of the different types of ML models to choose from, a Random Forest model was selected due to its lack of strict data assumptions and resilience to feature correlation.

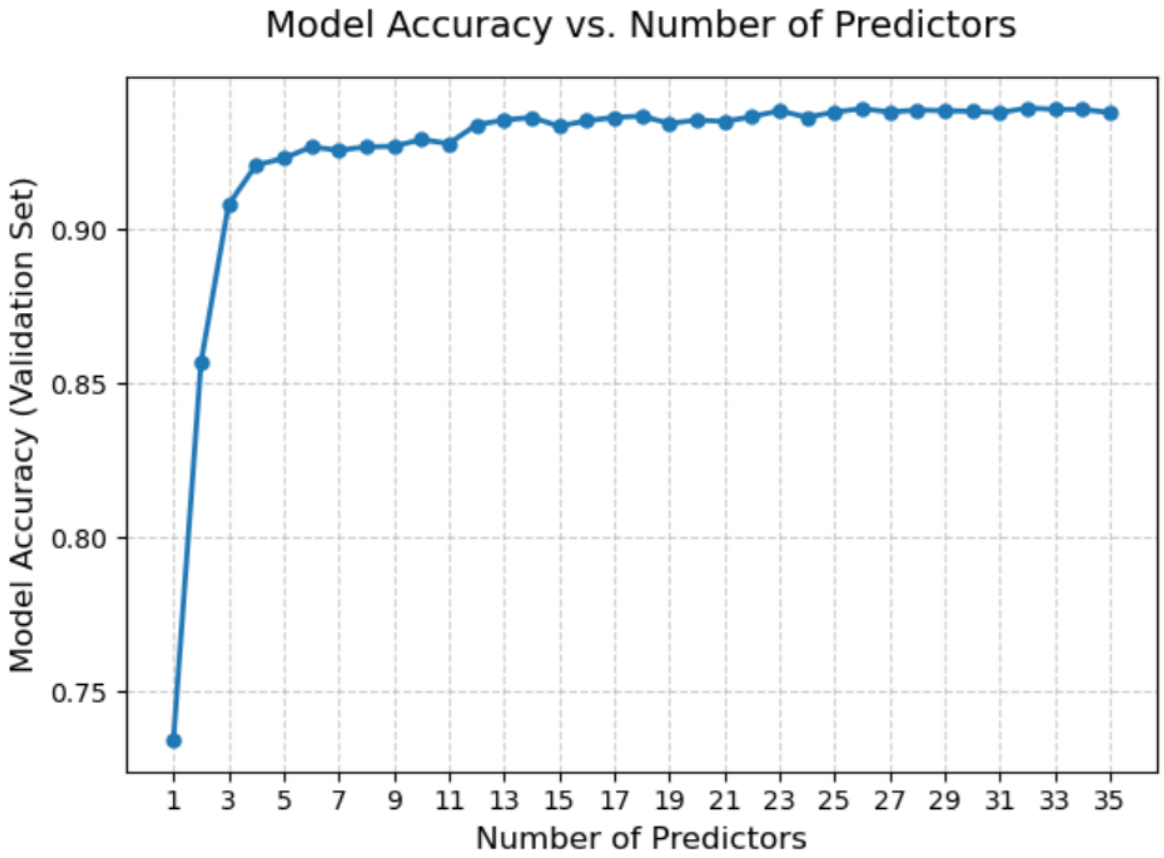
The initial model was trained using all 58 predictor variables, and one binary target variable (Job Success or Job Failure). The results were astonishing — the model was able to predict the outcome of the jobs in the validation dataset with a 93% overall accuracy! This was especially encouraging because the training dataset contained an even split of 50% successes and 50% failures, so the accuracy number was not inflated by simply guessing "success" every time. Clearly, there are strong signals contained in the time series data for these jobs within the first 2 hours of runtime.

The feature importance scores confirmed that features based on the CPU time series data were by far the most predictive when compared to the GPU-specific features. To be sure, a second model was trained on only jobs which utilized GPU's, and even that model confirmed that it was the CPU data that determined whether a job would crash. There was no disappointment that all the time spent engineering those GPU variables was for naught, because an overall accuracy of 93% was a massive result.

However, a random forest model that utilizes 58 predictor variables is complex and difficult to interpret, so testing continued to determine if the model could be simplified without sacrificing accuracy. Features that were less predictive than a random variable were removed, as were features that were over 90% correlated with another, more important feature. This reduced the total to 35 features, and after re-training the model, it still achieved a 93% accuracy score.

To reduce the number of predictors further, a loop was engineered in the Python code to train 35 separate models, where the first model would only have one predictor variable, and the last model would have all 35. This allowed for the identification of a point of diminishing accuracy returns as the number of features increased. The key finding was that

with only 4 features, the model achieved a 92% accuracy score, so this became the chosen configuration for the final model to achieve a good balance of accuracy and simplicity.

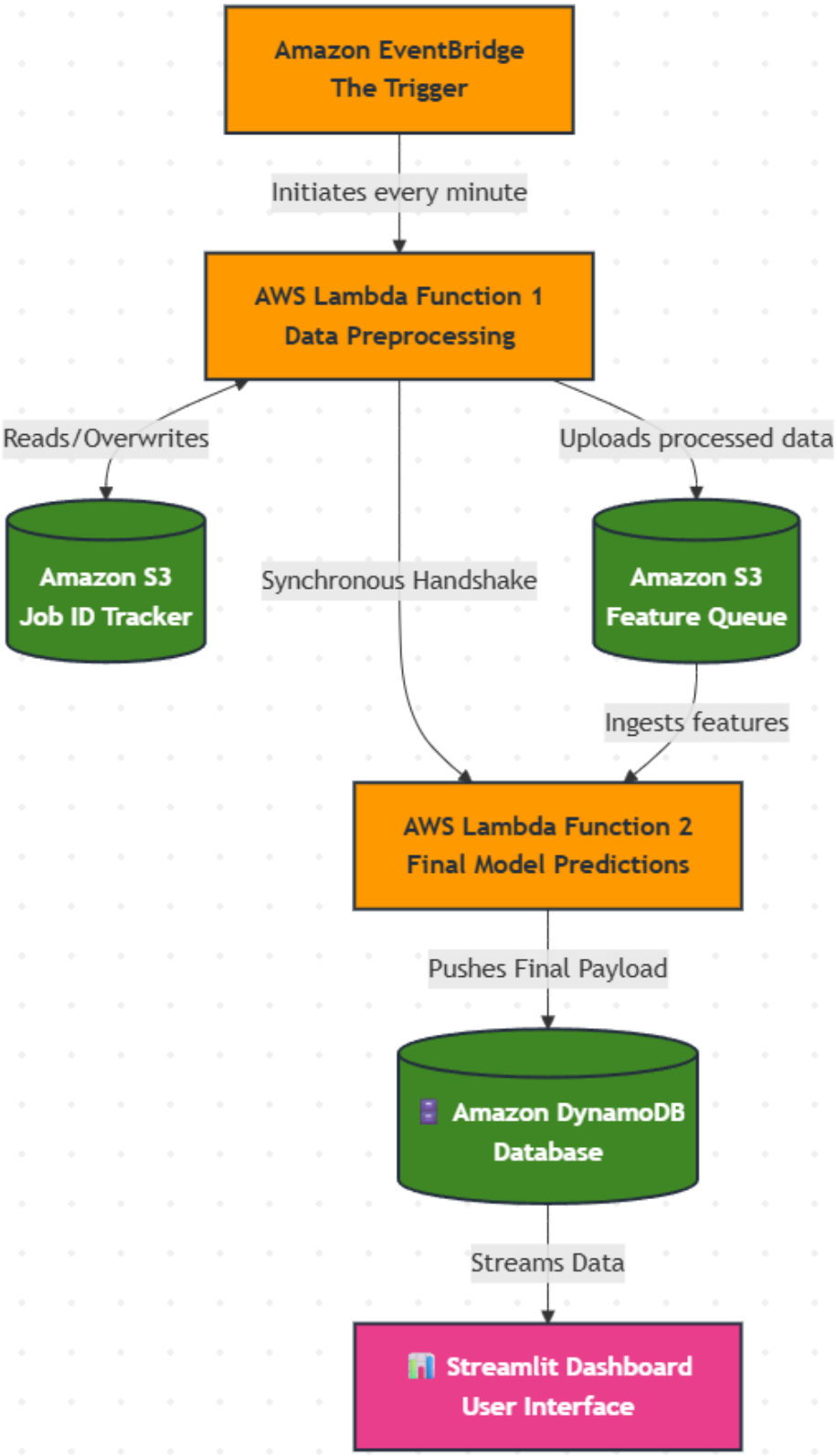


After selecting the final four predictor variables, some last-minute model tuning was performed to create the final model. When predicting the outcome on the "test" dataset that had not been seen by any previous model, the final model scored an overall accuracy of 92.4%. While this was not quite as high as the initial model with all 58 features, it was still an excellent result, and much easier to interpret. For those curious about which features made the final cut, that information can be found on the project's GitHub page in the file "Step6_Modeling_&_Analytics.ipynb".

Model Deployment

To simulate a real-world deployment of the final model, an AWS workflow was created that can process raw data directly from the MIT Supercloud S3 bucket, generate a prediction, and push the results to a live web dashboard. In the original MIT dataset, a new job was starting once every minute on average, so this workflow was designed to select a random job every minute for processing. In a true production deployment, the workflow would be configured to select a job as soon as it hit the 2-hour runtime mark and automatically stop the job if the prediction is a failure.

AWS was chosen as the deployment platform for the final model because the source data is stored in AWS, and it was possible to take advantage of the free-tier allowances for AWS Lambda, S3, ECR, and DynamoDB. Deploying to other platforms such as Databricks would have incurred more charges for being "always on", making AWS the straightforward choice. Below is a visualization of how the workflow is currently set up:


Amazon EventBridge
The Trigger
Initiates every minute
AWS Lambda Function 1
Data Preprocessing
Reads/Overwrites
Uploads processed data
Amazon S3
Job ID Tracker
Synchronous Handshake
Amazon S3
Feature Queue
Ingests features
AWS Lambda Function 2
Final Model Predictions
Pushes Final Payload
Amazon DynamoDB
Database
Streams Data
Streamlit Dashboard
User Interface

The first step was to create a Simple Storage Service (S3) bucket and upload a table containing all 125,000 jobs in the original "over two hour" subset. This table also contained metadata about each job, and a unique link to the MIT S3 folder for each job, which was appended using Databricks. Next, an AWS Lambda function was created to select a random job from the table in the S3 bucket and use the MIT S3 link to import the CPU timeseries log for that job. The function then processed the log to calculate the four feature values needed for the random forest model and exported this data as a new CSV file to a folder in the S3 bucket.

Once the first Lambda function finished running, it was configured to automatically trigger a second Lambda function ("Function 2"). Function 2 contained the random forest model and had to be created via a Docker container because it required the scikit-learn library to work. The second function proceeded to ingest the most recent CSV file in the "Processed Jobs" folder of the S3 bucket and run the four feature values through the 100 decision trees contained in the random forest model. The two outputs were the prediction (Completed or Failure), as well as the confidence percentage. Confidence in this context is the percentage of decision trees that "voted" for one outcome. The decision threshold was 50%, so if the model had 51% confidence in a job failure, the decision would be job failure, and vice versa for a completed job.

Those two outputs were then placed onto a DynamoDB table, which a live Streamlit dashboard is "pointed" to. When viewing the dashboard, it will be noticed that 1 prediction is made every hour. This is because the AWS workflow begins to cost a small amount of money when making 1 request per minute for an entire month. Since remaining within the free tier was a priority for this project, the job cadence was changed to 1 per hour, even though the workflow is physically able to handle 1 request per minute. The link to the live dashboard can be found on the project's GitHub page, and viewing on a desktop is recommended, as it is not optimized for mobile devices.

Business Impact

Since data is available about the duration of every job in the MIT Supercloud dataset that failed, it is possible to obtain a rough estimate of how many total hours of electricity could be saved by prematurely terminating jobs that are likely to crash at the 2-hour mark. With a 92% accurate model, the total number of runtime hours that could have been saved over the 9-month recording period from only jobs that failed is 278,000 hours.

The model was also used to predict job failures for a sample of jobs that were prematurely cancelled by the user to get an idea of the potential "cancelled job" failure rate. To do this, a random sample of 5,000 cancelled jobs was processed via Databricks, and the model predicted them to fail at a rate of 36%. When extrapolating this percentage to the entire pool of cancelled jobs, and accounting for the fact that cancelled jobs have an average duration of 46 hours, this saves an extra 306,000 compute hours. Adjusting the estimates from 9-months to 1 year brings the grand total to 780,000 compute hours potentially saved over the course of a year for all jobs.

It is difficult to determine an exact number for the average amount of electricity one job would consume in an hour, so a conservative estimate of 0.5 kWh of electricity at a cost of $0.10 per kWh was adopted. After performing the calculations, this results in 390,000 kWh of electricity saved, which equates to saving $39,000 over the course of a year. For context, this is the equivalent of powering 35 homes for 1 year or saving approximately 110 metric tons of carbon dioxide emissions. Again, these numbers constitute an estimate and could be different in a production setting due to several factors such as changing job durations and energy costs.

Going Forward

Even though this project was completed for all intents and purposes when the final model was deployed, there are still many further optimizations that can be made by analyzing the MIT dataset. Things that were left on the table include; an analysis of jobs that ran for less than 2 hours, predicting datacenter node failures, classifying jobs by workload type, and many more. There are also other types of ML models that could be applied to the dataset to attempt to boost results, such as a Neural Network model.

To put this project into perspective, MIT's datacenter is orders of magnitude smaller than a single modern commercial datacenter, and there are over 500 new datacenters currently under construction in the US alone. While it is likely that commercial datacenter operators are actively trying to minimize job crashes more than MIT might be, I would welcome further conversations with anyone who has industry knowledge about datacenters to broaden my understanding of the subject.

Lastly, it is worth pointing out that it is possible to change the decision threshold of the final model to adjust the prediction precision. For example, if the model required a confidence score of 80% rather than 50% to make a "job failure" prediction, this would reduce the number of false positives dramatically. The tradeoff would be that far fewer job failures would be caught, resulting in less compute hours saved overall. In a production datacenter deployment, it could make sense to start out with a higher threshold value for predicting job failures. This way the effectiveness of the model could be evaluated during a "pilot period", before adjusting it at a later time.

Conclusion

I'd like to wrap up this report by saying that this project was surprisingly fun to work on and felt like solving a large/complicated puzzle. It made me dig back in my notes from graduate school and figure out how to apply data science concepts to the real world. I also got a chance to work with industry-leading platforms such as Databricks and AWS for the first time in earnest. It was enjoyable and rewarding to think critically about how I wanted to approach this project and then re-adjust as I learned new information.

I was excited to find patterns in the MIT Supercloud dataset that were able to accurately inform predictions. I had no expectations that this was going to be the case, since at the beginning of the project I had little domain knowledge about datacenters. But my hunch turned out to be well founded, and I look forward to applying the skills I learned from this project to more contemporary problems in the future.